Powered by mindgrasp

Note 1

Network Programming

Network programming involves writing software that enables communication between two or more computers or processes over a network. **Examples include:**

- A web browser communicating with a web server.
- A mobile application interacting with an application server.
- A chat application facilitating communication between clients through a server.

In this experiment, a Client communicates with a Server, utilizing the `java.net` package for networking functionality.

Note 2

Important Networking Terms

Before diving into the program, it's crucial to understand the following terms:

- **Client:** A program that requests a service from another program. For example, a client might send "Hello Server."
- **Server:** A program that waits for requests from clients and provides services. For instance, a server might respond with "Hello Client."
- **IP Address:** Identifies a computer or device on a network (e.g., 192.168.1.10, 127.0.0.1).
- **Port Number:** Identifies a specific application or service running on a computer. In this program, the server listens on port **5000**.
- **Socket:** An endpoint for communication between two programs. The client and server communicate through sockets.

Conceptual Communication Flow:

CLIENT SERVER

<----- Connection Accepted ----
------ Hello Server ---------->
<----- Hello Client ----------
------ Close ---------------->

Note 3

TCP vs UDP

This experiment uses **TCP (Transmission Control Protocol)**, which is:

- Connection-oriented

- Reliable
- Ordered
- Error-controlled

Java uses **Socket** and **ServerSocket** for TCP communication.
In contrast, **UDP (User Datagram Protocol)** is:

- Connectionless
- Faster
- Less reliable
- Message/datagram based

Java provides **DatagramSocket** and **DatagramPacket** for UDP.
Important Exam Differences:

TCP	UDP
Connection-oriented	Connectionless
Reliable	Less reliable
Ordered data	No guarantee of order
Uses Socket/ServerSocket	Uses DatagramSocket
More overhead	Less overhead

Note 4

Packages Used in the Program

The program utilizes the following packages:

- **java.io:** Provides classes for input and output operations (e.g., BufferedReader, InputStreamReader, PrintWriter).
- **java.net:** Provides classes for network programming (e.g., Socket, ServerSocket).

Note 5

Server Program

The correct import statement should be:

```java
import java.io.*;
import java.net.*;
```

Complete Server Code:

```java
import java.io.*;
import java.net.*;
public class Server {
public static void main(String[] args) {
try {
```

```
ServerSocket serverSocket = new ServerSocket(5000);
System.out.println("Server started...");
System.out.println("Waiting for client...");
Socket socket = serverSocket.accept();
System.out.println("Client connected.");
BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
String message = input.readLine();
System.out.println("Client says: " + message);
output.println("Hello Client");
socket.close();
serverSocket.close();
System.out.println("Connection closed.");
} catch (IOException e) {
System.out.println(e);
}
}
}
```

Note 6

Client Program

Complete Client Code:

```java
import java.io.*;
import java.net.*;
public class Client {
public static void main(String[] args) {
try {
Socket socket = new Socket("localhost", 5000);
System.out.println("Connected to server.");
BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
output.println("Hello Server");
String response = input.readLine();
System.out.println("Server says: " + response);
socket.close();
System.out.println("Connection closed.");
} catch (IOException e) {
System.out.println(e);
}
}
}
```

Note 7

Why Are Server and Client Separate Programs?

The server and client are separate programs because they represent different processes.

- The server waits for the client.
- The client connects to the server.

They should be compiled separately:

- **Server.java → Server.class**
- **Client.java → Client.class**

Note 8

Server Program — Line-by-Line Explanation

Line 1

```java
import java.io.*;
```

Imports classes required for input/output operations.

Line 2

```java
import java.net.*;
```

Imports networking classes, allowing the use of **ServerSocket** and **Socket**.

Server Class

```java
public class Server
```

Defines a class named Server. The file should be named **Server.java**.

Main Method

```java
public static void main(String[] args)
```

This is the entry point of the Java application.

Why try-catch?

Networking operations can cause exceptions (e.g., port already occupied, server unreachable). The **IOException** is used to handle these exceptions.

Creating the ServerSocket

```java
ServerSocket serverSocket = new ServerSocket(5000);
```

This statement creates a ServerSocket that listens for incoming TCP connections on port **5000**.

accept() — Most Important Server Method

```java
Socket socket = serverSocket.accept();
```

This line waits for an incoming client connection. It is a blocking operation, meaning the server will wait here until a client connects.

What Does accept() Return?

The **accept()** method returns a **Socket** object representing the communication connection

between the server and the client.

Note 9

Conclusion

This document provides a detailed overview of a basic **TCP Client-Server program** in Java, covering essential concepts, networking terms, and a thorough explanation of the code. Understanding these principles is crucial for mastering network programming in Java and succeeding in your Advanced Java Lab syllabus.